Skip to content
© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2022
V. Sukhija, *PowerShell Fast Track*
https://doi.org/10.1007/978-1-4842-7759-1_1

1. PowerShell Basics

WARNING This material has been reproduced and communicated to you by or on behalf of the CQUniversity, Australia pursuant to Part IVA Division 4 of the Copyright Act 1968 (the Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act. Do not remove this notice. Sukhija, V 2021 *PowerShell Fast Track: Hacks for Non-Coders.* Berkeley, CA: Apress L. P. https://cqu.primo.exlibrisgroup.com/permalink/61CQU_INST/669qn3/cdi_safari_books_v2_9781484277591

Vikas Sukhija[1]
(1) Waterloo, ON, Canada

Let's start with basic elements and quickly review variables, loops, `if/else` statements, switches, and functions. These are the heart of any scripting language, and will assist you in creating simple to complex scripts.

I will not delve into PowerShell versions or what PowerShell is. (But here is an easy-to-understand definition without going into depth: PowerShell is a task automation solution made up of a command-line shell and a scripting language.) I also won't talk about what platforms it can be used on or get and set commands, because this is not an in-depth book for learning the language.

The intent of this book is to teach you how to create scripts without having a deep knowledge of under-the-hood elements. This is an approach I've used successfully with many students. They gradually became well-versed with the language.

Not everyone is from a programming background and not everyone is adept at creating code. However, by following the approach described in this book, you will be able to quickly create your own scripts and automate IT systems/processes.

> **Note** All source code used in the book can be accessed by clicking the **Download Source Code** button located at `www.apress.com/9781484277584` (follow the listing numbers).

Variables and Printing

To begin, you need to understand the basics, which include variables and arrays. Every variable in PowerShell starts with a dollar sign ($), such as

```
$a = "1"
$b = "Vikas"
```

When you type `$a` and `$b`, values will be displayed, as shown in Figure 1-1.

```
Windows PowerShell
PS C:\> $a="1"
PS C:\> $b="vikas"
PS C:\> $a
1
PS C:\> $b
vikas
PS C:\>
```

Figure 1-1 Variables in PowerShell

Now you can use `Write-host` to print this to the screen:

```
Input: PS C:\> Write-host $a
Output: 1
Input: PS C:\> Write-host $b
Output: vikas
```

> **Tip** Make it a rule to use quotes when assigning values to variables, as shown above in the two examples.

Let's change the foreground color of what is being displayed by `Write-host`:

```
Input: PS C:\> Write-host $b -ForegroundColor Green
Output: vikas
Input: PS C:\> Write-host "processing ........." -ForegroundColor
Green
Output: processing .........
PS C:\>
```

Figure 1-2 shows the results.

```
Windows PowerShell
PS C:\> Write-host $a
1
PS C:\> Write-host $b
vikas
PS C:\> Write-host $b -ForegroundColor Green
vikas
PS C:\> Write-host "processing ........." -ForegroundColor Green
processing .........
PS C:\>
```

Figure 1-2 Using the Write-host variable

Let's illustrate arrays quickly. In PowerShell, arrays can be defined in the same way as variables. Here are some examples:

```
$b = "A","B","C","D","E"
```

If you define it as a variable and separate the elements by a comma, PowerShell automatically understands that it is an array, as shown in Figure 1-3.

```
PS C:\>  $b = "A","B","C","D","E"
PS C:\> $b
A
B
C
D
E
```

Figure 1-3 Array illustration

The proper way of defining an array is as follows because it is understood from the syntax itself that it is an array (as shown in Figure 1-4):

```
$c= @("server1","server2")
```

```
Windows PowerShell
PS C:\> $c= @("server1","server2")
PS C:\> $c
server1
server2
```

Figure 1-4 Array syntax

A dynamic array can be defined as `@()`. You will use this type in examples in this book to add elements in the array and generate reports:

```
$d = @()
```

If/Else Switch

`If else` is condition-based processing. It is the basis of any scripting language. If some condition is true, you need to process something; otherwise, process some other thing.

Listing 1-1 shows two examples. First, you define a variable value as 10 and then you use the conditional operators and `if else` statements to check if it's greater than 9 or if it's less than 9. Based on the result, you use `Write-host` to print it to screen as shown in Figure 1-5.

Note that `-gt` means greater than and `-lt` means less than. Do not worry; I will quickly go through them in the next subsection.

```
[int]$a= "10"
if($a -gt "9")
{
write-host "True" -foregroundcolor Green
```

```
}else {
Write-host "False" -foregroundcolor Red
}
```

Listing 1-1 Example Code for Greater Than Operator Usage in If/Else

Yes, [int] that means integer. If you use a prefix before the variable, it means you have exclusively defined it as an integer. Defining it is always better but if you don't, PowerShell is intelligent enough to do it implicitly. These are called datatypes, and they include [string], [char], [int], [array], etc.

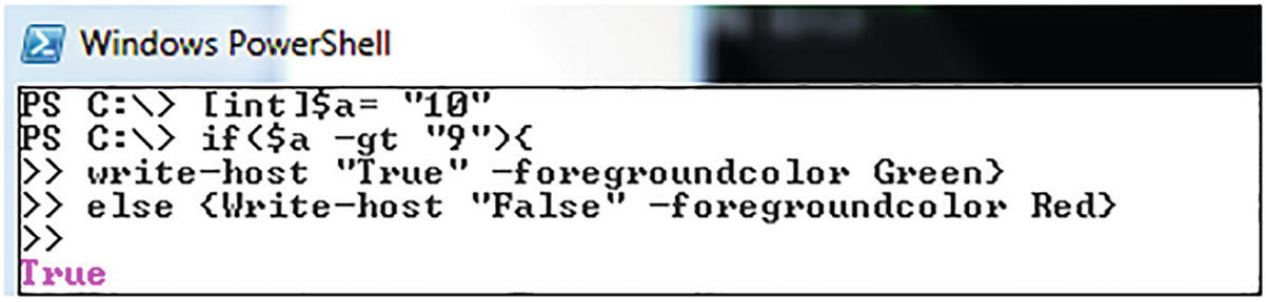

Figure 1-5 Showing -gt usage in if/else

Listing 1-2 and Figure 1-6 show a less than operator usage snippet.

```
[int]$a= "10"
if($a -lt "9"){
write-host "True" -foregroundcolor Green
}else {
Write-host "False" -foregroundcolor Red
}
```

Listing 1-2 Example Code for the Less Than Operator Usage in If/Else

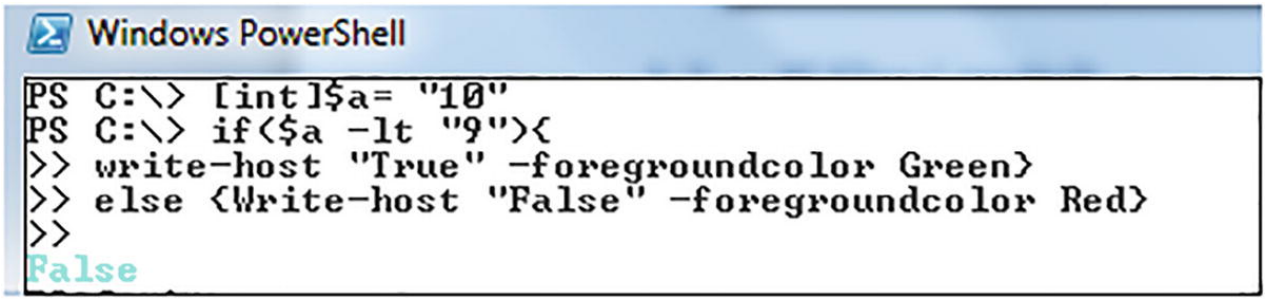

Figure 1-6 Showing -lt usage in if/else

Conditional/Logical Operators

Below is a list of conditional/logical operators that you will use in your everyday scripts. Without them, many scripting operations would not be possible. They will always be used in comparison `if else` statements as shown in the above parent section.

- `-eq` Equal
- `-ne` Not equal
- `-ge` Greater than or equal
- `-gt` Greater than

- `-lt` Less than
- `-le` Less than or equal
- `-like` Wildcard comparison
- `-notlike` Wildcard comparison
- `-match` Regular expression comparison
- `-notmatch` Regular expression comparison
- `-replace` Replace operator
- `-contains` Containment operator
- `-notcontains` Containment operator

Logical Operators

- `-and` Logical AND
- `-or` Logical OR
- `-not` Logical NOT
- `!` Logical NOT

Logical operators are used when you want to combine the conditions. Let's update the above example to the code shown in Listing 1-3. You will print true if the value of variable `$a` is less than 9 or equals to 10. Here you have combined two conditions. Since it is the `OR` operator, `TRUE` will be returned if one of them matches. Here the second condition, `$a -eq "10"`, matches if a value is equal to 10. See the results in Figure 1-7.

```
[int]$a= "10"
If(($a -lt "9") -or ($a -eq "10")){
write-host "True" -foregroundcolor Green
}else {
Write-host "False" -foregroundcolor Red}
```

Listing 1-3 Example Code Showing Logical -or Operator

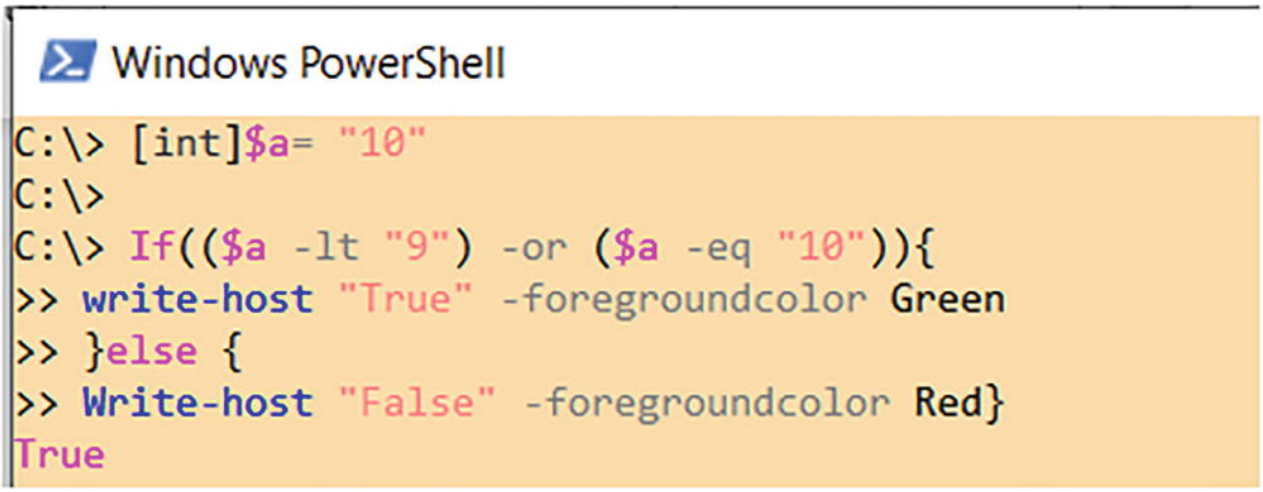

Figure 1-7 Showing logical -or operator

If you use the `AND` operator, then both conditions should match if you want to return `TRUE`, which will not happen in the above case. See Listing 1-4 and Figure 1-8.

```
[int]$a= "10"
If(($a -lt "9") -and ($a -eq "10")){
write-host "True" -foregroundcolor Green
}else {
Write-host "False" -foregroundcolor Red}
```

Listing 1-4 Code Showing Logical -and Operator

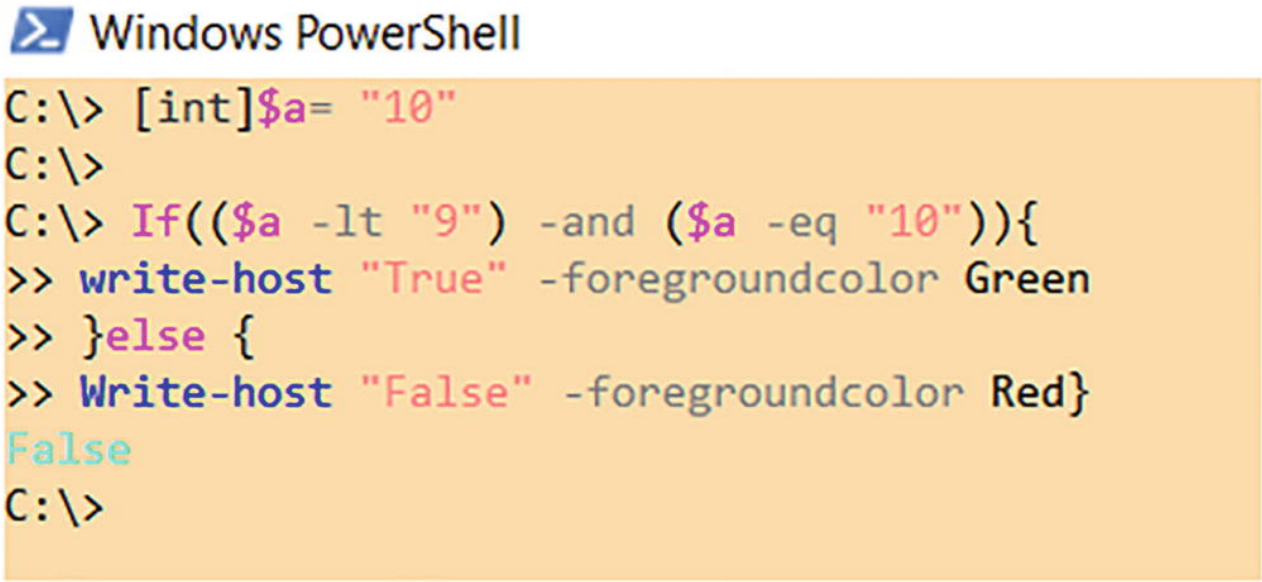

Figure 1-8 Showing logical -and operator

Not is for negation. I generally don't use it often. In my experience, it causes human error if you are in hurry and do not think it through enough.

Loops

There are two main loops in any scripting language and that is true for PowerShell as well. There are others but they are all variants of these two.

For Loop and While Loop

For Loop

There are three iterations of `for` loops in PowerShell:

- `foreach`
- `foreach-object`
- `for`

Let's differentiate between the three `for` loops by looking at the examples.

`foreach`: You need to specify a `foreach $variable` in `$collection`: `foreach ($i in $x)`.

Note You combine the `if else` and comparison operators in Listing 1-5. You can see the results in Figure 1-9.

```
$x=@("1","2","3",,"4")
foreach ($i in $x) {
      if ($i -lt 2) { write-host "$i is Green" -foregroundcolor Green
      }
         else{ write-host "$i is yellow" -foregroundcolor yellow
      }
 }
```

Listing 1-5 Code Showing a foreach Loop

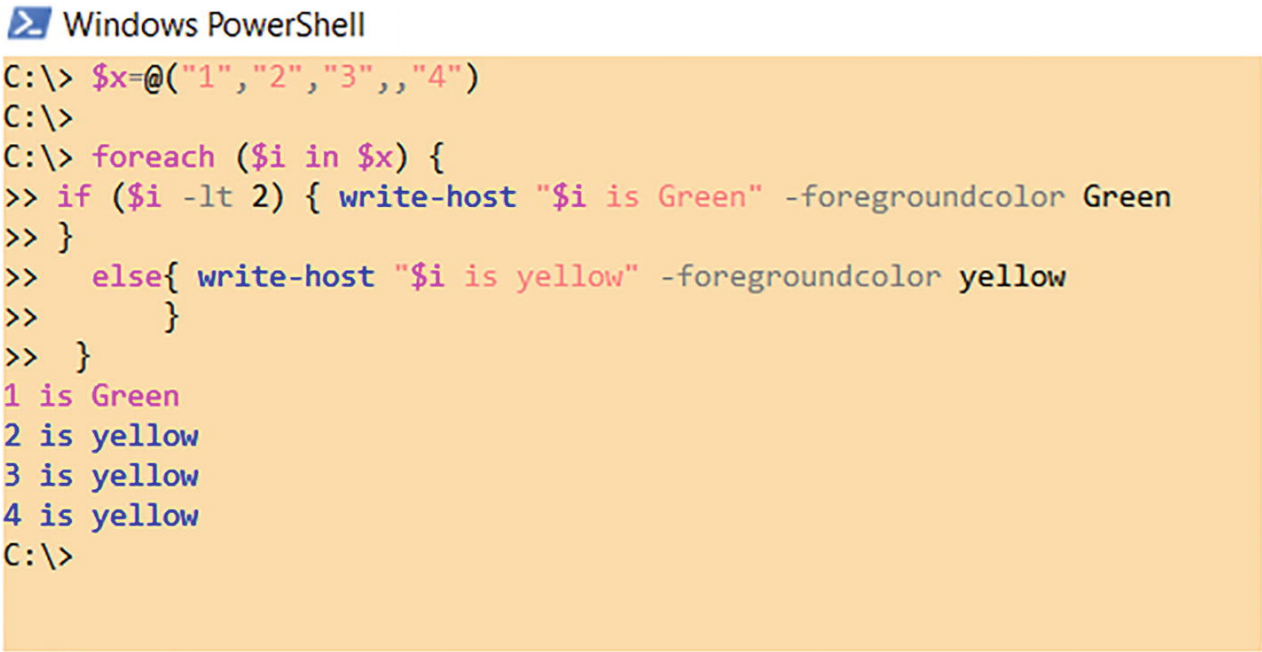

Figure 1-9 Showing a foreach loop

`foreach-object`: You use a PIPE with the collection to achieve the same thing (see Listing 1-6 and Figure 1-10):

```
   $x | foreach-object
$x=@("1","2","3",,"4")
$x | foreach-object{
        if ($_ -lt 2) { write-host "$_ is Green" -foregroundcolor
Green
}
        else{ write-host "$_ is yellow" -foregroundcolor yellow
}
     }
```

Listing 1-6 Code Showing a foreach-object Loop

```
Windows PowerShell
C:\> $x=@("1","2","3",,"4")
C:\>
C:\> $x | foreach-object{
>>   if ($_ -lt 2) { write-host "$_ is Green" -foregroundcolor Green
>> }
>>   else{ write-host "$_ is yellow" -foregroundcolor yellow
>> }
>>      }
1 is Green
2 is yellow
3 is yellow
4 is yellow
C:\>
```

Figure 1-10 Showing a foreach-object loop

`for`: This is the one you will remember from your school days. I have not used it much and see less usage across the community. See the code in Listing 1-7 and the results in Figure 1-11.

```
for($x=1; $x -le 5; $x++){
    if($x -lt 2){write-host "$x is Green" -foregroundcolor Green
    }
    else{ write-host "$x is yellow" -foregroundcolor yellow
    }
 }
```

Listing 1-7 Code Showing a for Loop

```
Windows PowerShell
C:\> for($x=1; $x -le 5; $x++){
>> if($x -lt 2){write-host "$x is Green" -foregroundcolor Green
>> }
>>   else{ write-host "$x is yellow" -foregroundcolor yellow
>> }
>>  }
1 is Green
2 is yellow
3 is yellow
4 is yellow
5 is yellow
C:\>
```

Figure 1-11 Showing a for loop

While Loop

The `while` loop is different because it lasts until the condition is true. Let's go through some examples to get more clarity.

The `while` loop also has two iterations:

- `do-while`
- `while`

For `do-while` you do something until some condition is met. In Listing 1-8, variable `x = 0` and inside the variable you increment its value until it is not equal to 4. See Figure 1-12 for the result.

Note You are doing the thing first and matching the condition later.

```
$x= 0
Do {$x++
if($x -lt 2){write-host "$x is Green" -foregroundcolor Green
              }
             else{ write-host "$x is yellow" -foregroundcolor yellow
}
 }while($x -ne 4)
```

Listing 1-8 Code Showing a do-while Loop

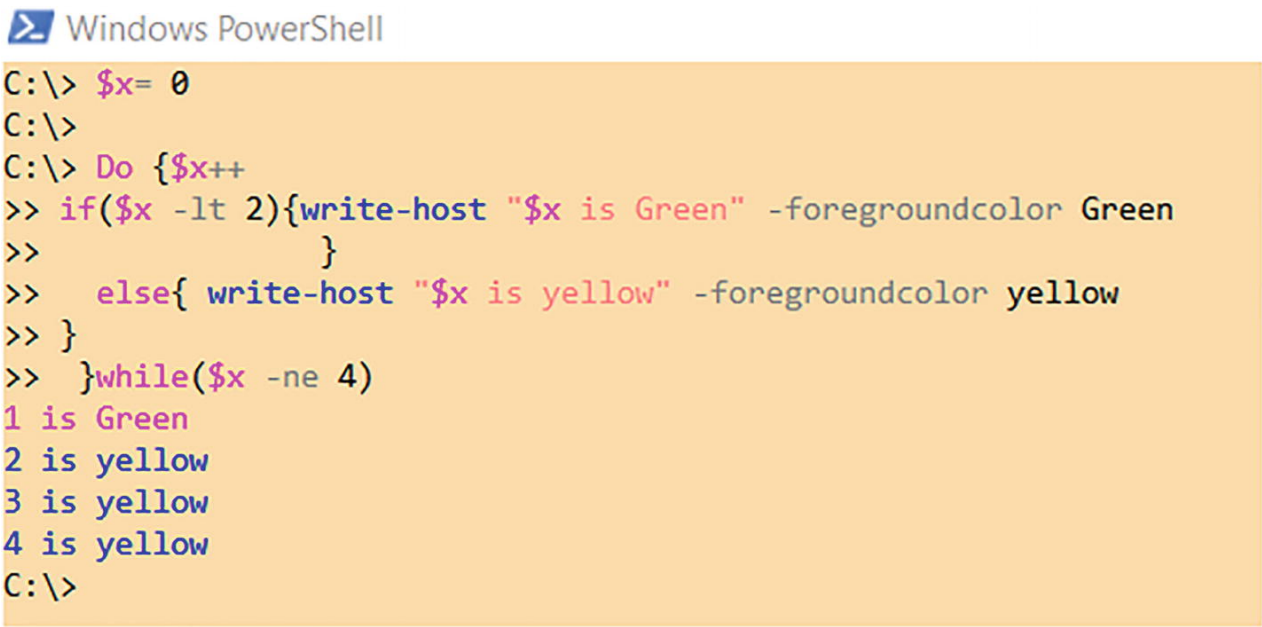

Figure 1-12 Showing a do-while loop

For `while, you are also` doing something until some condition is met. In Listing 1-9, variable `x = 0` and inside the variable you increment its value until it is not equal to 4.

Note You are checking first and doing the thing after that.

The main difference between the two, as you can see, is the `while` loop (an example of which is shown in Listing 1-9) checks the condition before the loop (iteration) but `do-while` does the checks after the execution. See Figure 1-13 for the result.

```
$x= 0
while($x -ne 4) {$x++
```

```
if($x -lt 2){write-host "$x is Green" -foregroundcolor Green
        }
        else{ write-host "$x is yellow" -foregroundcolor yellow
       }
}
```

Listing 1-9 Code Showing the while Loop

```
Windows PowerShell
C:\> $x= 0
C:\>
C:\> while($x -ne 4) {$x++
>> if($x -lt 2){write-host "$x is Green" -foregroundcolor Green
>> }
>> else{ write-host "$x is yellow" -foregroundcolor yellow
>> }
>> }
1 is Green
2 is yellow
3 is yellow
4 is yellow
C:\>
```

Figure 1-13 Showing a while loop

Functions

Functions are entities that, once defined, can be called anywhere in the script. Using functions avoids repetitive, lengthy code. Here's a glimpse for better understanding. I will not go in any details about parametrization and advanced functionalities of a function as my main motive here is a little bit of understanding and combining the code together to get the work done. In Listing 1-10, you create an `Add` function to add two numbers. The result is shown in Figure 1-14.

```
Function Add ($a1, $b1)
{
$a1 + $b1
}
```

Listing 1-10 Example Code Showing an Add Function of Two Numbers

Add 5 6 # Call function

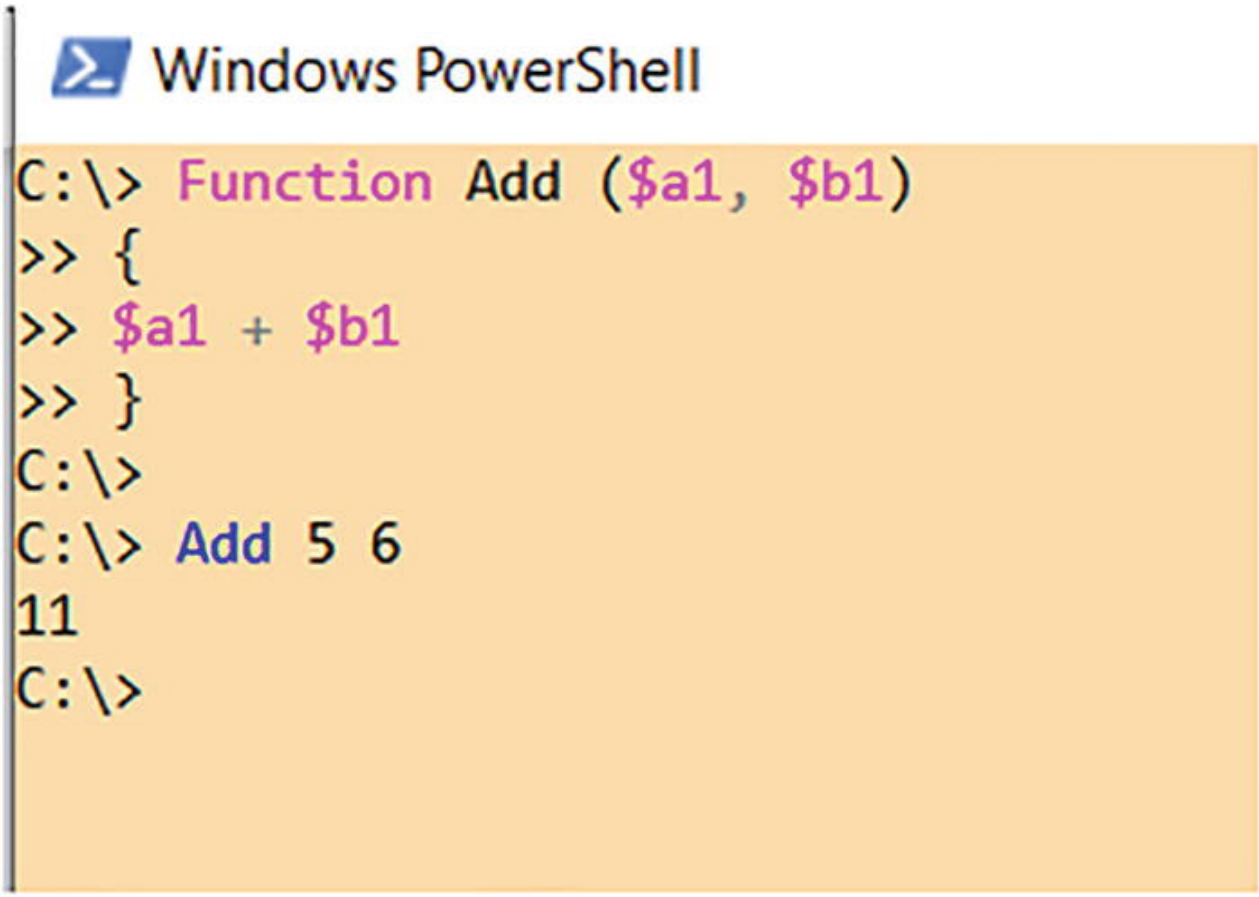

Figure 1-14 Showing an Add function of two numbers

Similarly, you can create this for three or more numbers. See Listing 1-11 and Figure 1-15.

```
Function Add ($a1, $b1, $c1)
{
$a1 + $b1 +$c1
}
```

Listing 1-11 Example Code Showing an Add Function of Three Numbers

Add 5 6 9 # Call function

```
Windows PowerShell
C:\> Function Add ($a1, $b1,$c1)
>> {
>> $a1 + $b1 +$c1
>> }
C:\>
C:\> Add 5 6 9 # Call function
20
C:\>
```

Figure 1-15 Showing an Add function of three numbers

Summary

In this chapter, you learned about PowerShell basics such as variables, arrays, if/else statements, and loops, which are the building blocks for creating powerful scripts in a production environment. These basics will be utilized further in the following chapters.

table of contents
Settings